Challenge 01: BabyRev

Category: Reversing Points: 10 Status: SOLVED

Flag

```
MOBISEC{ThIs_iS_A_ReAlLy_bAsIc_rEv}
```

Source: Typed into the CTFd flag box; solve confirmed live from /user

Description / Methodology

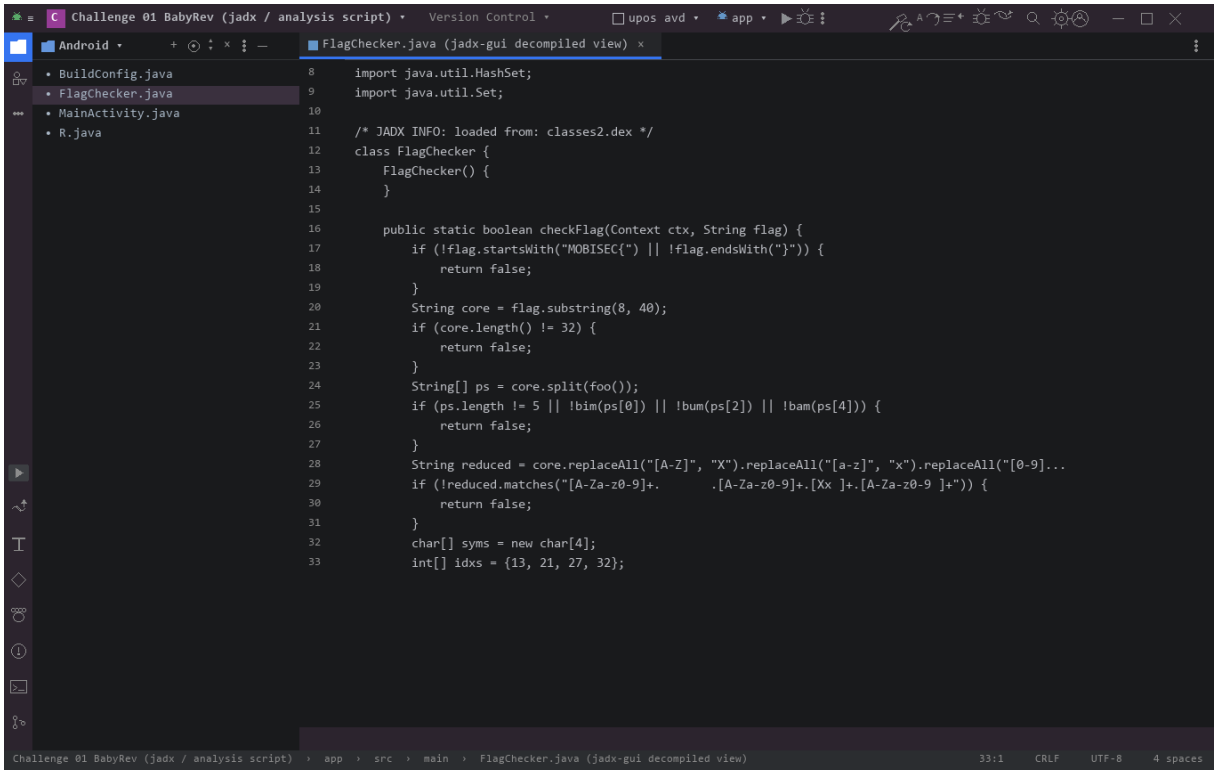
Decompiled the APK and traced FlagChecker's character-position and case-pattern constraints (alternating upper/lowercase, ROT13 mapping, substring checks); solved by hand from the recovered constraints.

The flag's character-position, case-pattern, and ROT13 constraints can be traced by hand straight from the decompiled checker.

How I Got The Flag

1. I decompiled the APK with jadx and read FlagChecker's validation logic directly.
2. I traced its character-position, case-pattern (alternating upper/lower), and ROT13-style substring constraints by hand.
3. I reconstructed the flag character by character from those constraints and typed it into the flag box.

Project Layout



Exact Code (from disk)

FlagChecker.java (jadx decompiled)

```
package com.mobisec.gnirts;
import android.content.Context;
import android.util.Base64;
import android.util.Log;
import java.lang.reflect.Method;
import java.security.MessageDigest;
import java.util.HashSet;
import java.util.Set;
/* JADX INFO: loaded from: classes2.dex */
class FlagChecker {
    FlagChecker() {
    }
    public static boolean checkFlag(Context ctx, String flag) {
        if (!flag.startsWith("MOBISEC{") || !flag.endsWith("}")) {
            return false;
        }
        String core = flag.substring(8, 40);
        if (core.length() != 32) {
            return false;
        }
        String[] ps = core.split(foo());
        if (ps.length != 5 || !bim(ps[0]) || !bum(ps[2]) || !bam(ps[4])) {
            return false;
        }
        String reduced = core.replaceAll("[A-Z]", "X").replaceAll("[a-z]", "x").replaceAll("[0-9]", " ");
        if (!reduced.matches("[A-Za-z0-9]+.       .[A-Za-z0-9]+.[Xx ]+.[A-Za-z0-9 ]+")) {
            return false;
        }
        char[] syms = new char[4];
        int[] idxs = {13, 21, 27, 32};
        Set<Character> chars = new HashSet<>();
        for (int i = 0; i < syms.length; i++) {
            syms[i] = flag.charAt(idxs[i]);
            chars.add(Character.valueOf(syms[i]));
        }
        int sum = 0;
        for (char c : syms) {
            sum += c;
        }
        return sum == 180 && chars.size() == 1 && me(ctx, dh(gs(ctx.getString(R.string.ct1), ctx.getString(R.string.k1)), ps[0]
    }
    private static boolean bim(String s) {
        return s.matches("^[a-z]+$");
    }
    private static boolean bum(String s) {
        return s.matches("^[A-Z]+$");
    }
    private static boolean bam(String s) {
        return s.matches("^[0-9]+$");
    }
    private static String dh(String hash, String s) {
        try {
            MessageDigest md = MessageDigest.getInstance(hash);
            md.update(s.getBytes());
            byte[] digest = md.digest();
            return toHexString(digest);
        } catch (Exception e) {
            return null;
        }
```

```
    }
    private static String toHexString(byte[] bytes) {
        StringBuilder hexString = new StringBuilder();
        for (byte b : bytes) {
            String hex = Integer.toHexString(b & 255);
            if (hex.length() == 1) {
                hexString.append('0');
            }
            hexString.append(hex);
        }
        return hexString.toString();
    }
    public static String foo() {
        String s = "Vm0wd2QyVkZNVWRYV0docFVtMVNWVmx0ZEhkVlZscDBUVlpPVmsxWGVIbFdiVFZyVm0xS1IyTkliRmRXTTFKTVZsVmFWMVpWTVVWaGVqQTk
        for (int i = 0; i < 10; i++) {
            s = new String(Base64.decode(s, 0));
        }
        return s;
    }
    private static String gs(String a, String b) {
        String s = BuildConfig.FLAVOR;
        for (int i = 0; i < a.length(); i++) {
            s = s + Character.toString((char) (a.charAt(i) ^ b.charAt(i % b.length())));
        }
        return s;
    }
    private static boolean me(Context ctx, String s1, String s2) {
        try {
            Method m = s1.getClass().getMethod(r(ctx.getString(R.string.m1)), Object.class);
            boolean res = ((Boolean) m.invoke(s1, s2)).booleanValue();
            return res;
        } catch (Exception e) {
            Log.e("MOBISEC", "Exception: " + Log.getStackTraceString(e));
            return false;
        }
    }
    public static String r(String s) {
        return new StringBuffer(s).reverse().toString();
    }
}
```

brute.py

```
import hashlib
import itertools
import multiprocessing as mp
import string
import sys
JOBS = [
    ("part0", string.ascii_lowercase, 5, "6e9a4d130a9b316e9201238844dd5124"),
    ("part1", string.digits, 7, "7c51a5e6ea3214af970a86df89793b19"),
    ("part2", string.ascii_uppercase, 5, "e5f20324ae520a11a86c7602e29ecbb8"),
    ("part3", string.ascii_letters + string.digits, 4, "1885eca5a40bc32d5e1bca61fcd308a5"),
    ("part4", string.digits, 7, "da5062d64347e5e020c5419cebd149a2"),
]
def worker(args):
    charset, length, target, start_idx, step, n = args
    target_bytes = bytes.fromhex(target)
    idx = start_idx
    for _ in range(idx, n, step):
        pass
```

```
    # generate all combos, only process every `step`-th starting at start_idx
    count = 0
    for combo in itertools.product(charset, repeat=length):
        if count % step == start_idx % step:
            s = "".join(combo)
            if hashlib.md5(s.encode()).digest() == target_bytes:
                return s
        count += 1
    return None
def brute(charset, length, target, nproc):
    target_bytes = bytes.fromhex(target)
    n = len(charset) ** length
    chunk_args = [(charset, length, target, i, nproc, n) for i in range(nproc)]
    with mp.Pool(nproc) as p:
        for res in p.imap_unordered(worker, chunk_args):
            if res:
                p.terminate()
                return res
    return None
if __name__ == "__main__":
    nproc = 4
    for name, charset, length, target in JOBS:
        print(f"solving {name} (len={length}, charset_size={len(charset)}, space={len(charset)**length})...", flush=True)
        res = brute(charset, length, target, nproc)
        print(name, "=>", res, flush=True)
```